Section 09: Retrieval-Augmented Generation (RAG)

LangChain Framework Guide

March 2026 — LangChain v1.2.x

Agenda

- RAG Mental Model: Giving LLMs access to your data
- RAG Fundamentals: Definition, motivation, pipeline
- Building RAG with LCEL: Complete working examples
- Advanced RAG Techniques: Multi-query, conversational, citations
- Evaluation and Production: Measuring quality, deployment checklist
- Best Practices: Common pitfalls, RAG vs fine-tuning

Goal: Master practical RAG implementation from basics to production.

Mental Model: RAG = Access to Your Data

Core Insight

RAG gives LLMs access to external knowledge without retraining. Think: "LLM + Search + Context Injection"

Without RAG:

- LLM knowledge frozen at training time
- Cannot answer questions about proprietary data
- Must fine-tune to add new knowledge

With RAG:

- Query retrieves relevant documents at runtime
- Context injected into prompt
- LLM answers based on retrieved docs

What is RAG? Definition and Motivation

Retrieval-Augmented Generation (RAG): A technique combining information retrieval with generative AI to produce answers grounded in external documents.

Key Advantages:

- No retraining needed
- Handles proprietary data
- Reduces hallucination
- Transparent sources
- Easy to update knowledge

Why RAG Over Fine-tuning:

- No GPU training required
- Knowledge stays current
- Can cite sources
- Works with large datasets
- Lower cost and faster deployment

The RAG Pipeline: Three Steps

1. Index Split documents into chunks, embed, store in vector DB

Retrieve Find K most relevant chunks for query

Generate Pass query with context to LLM, get answer

Documents → Chunks → Embeddings → Vector DB

Query → Top-K Retrieval → LLM → Answer

[Docs: docs.smith.langchain.com/rag]

Building a Basic RAG Chain (LCEL)

```
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings, ChatOpenAI
from langchain_community.vectorstores import Chroma
from langchain.chains import create_retrieval_chain
documents = load_documents("./docs")
splitter = RecursiveCharacterTextSplitter(chunk_size=1000)
chunks = splitter.split_documents(documents)
embeddings = OpenAIEmbeddings()
vector_db = Chroma.from_documents(chunks, embeddings)
retriever = vector_db.as_retriever(k=3)
llm = ChatOpenAI(model="gpt-4")
rag_chain = create_retrieval_chain(retriever, doc_chain)
result = rag_chain.invoke({"input": "What is RAG?"})
```

[Docs: LangChain LCEL docs]

The RAG Prompt: Formatting Context

Best Practice

Tell the LLM explicitly to use context and cite sources.

Good RAG Prompt Template:

- Clear instruction to use context
- Role/expertise framing
- Clear separation: context, query, answer
- Fallback behavior for missing context

Example Prompt: "You are an expert assistant. Answer based ONLY on the provided context. If context doesn't contain the answer, say 'Not in context.' Context: {context} Question: {input} Answer:"

create_stuff_documents_chain

Stuff Strategy

Combine all retrieved docs into one prompt. Good for small/medium contexts.

How it works:

1. Retriever returns K documents
2. All documents passed as single context block
3. LLM generates response based on context
4. Simple, effective, and fast

Advantages:

- Minimal latency
- Good for most use cases
- Easy to implement

Limitations:

- Context window can fill quickly with large docs
- No ranking or compression of results

create_retrieval_chain: Full RAG

Combines retriever + document chain into single pipeline.

```
 from langchain.chains import create_retrieval_chain
from langchain_openai import OpenAIEmbeddings
embeddings = OpenAIEmbeddings()
db = FAISS.load_local("vector_db", embeddings)
retriever = db.as_retriever(k=4)
llm = ChatOpenAI()
rag_chain = create_retrieval_chain(retriever, doc_chain)
result = rag_chain.invoke({"input":  "How does RAG work?"})
print(result["answer"])

print(result["context"])
```

Conversational RAG: Adding Chat History

Challenge

Standard RAG treats each query independently. Need to reformulate queries using conversation history.

Problem Example:

1. User: "Tell me about RAG"
2. User: "What about fine-tuning?"
3. Without context, system retrieves about fine-tuning in general
4. With context, system retrieves about fine-tuning vs RAG

Solution

Use `create_history_aware_retriever` to reformulate queries with chat history.

Multi-Query RAG: Multiple Perspectives

Intuition

One query formulation may miss relevant docs. Generate multiple query variations for better recall.

Standard Query:

- Single embedding
- May miss synonyms
- Limited perspective

Multi-Query:

- Generate 3-5 query variants
- Retrieve from each
- Combine and deduplicate
- Better coverage

Example Variants for "How do I build RAG?":

- "Steps to implement retrieval-augmented generation"
- "RAG architecture and setup guide"
- "Creating a RAG system from scratch"

RAG with Sources: Returning Retrieved Docs

Transparency

Include source documents in response so users can verify answers.

Output Structure:

- Answer: "RAG is a technique that..."
- Context/Sources:
  - docs/rag_intro.pdf page 2
  - docs/langchain_guide.md section 5
  - docs/tutorials.html

Benefits:

- Users can verify answers
- Build trust in system
- Easy debugging

RAG with Citations: Making LLM Cite Passages

Goal

Have LLM cite specific passages from retrieved documents.

Citation Prompt Instruction:

- "Include [Citation: source_name, passage] when citing"
- "For each claim, provide the source document"
- "Format as [Source: filename, section X]"

Example Output:

- "RAG is [Citation: langchain_docs, section 2.1] a technique that combines retrieval with generation."

Advantages:

- Verifiable answers
- Transparent reasoning
- Reduced hallucination

Contextual Compression: Filter for Relevance

Problem

Retrieved chunks may contain irrelevant sections, wasting context window and confusing LLM.

Solution

Use LLM or embedding-based filter to keep only relevant passages.

Approaches:

1. LLM-based: Use LLM to score and filter documents
2. Embedding-based: Compute similarity to query
3. Keyword matching: Keep paragraphs with query terms

Benefits:

- Fit more documents in context window
- Reduce LLM confusion
- Lower token usage and costs

Parent Document Retriever: Chunk vs. Context

Idea

Index small chunks for fine-grained retrieval, but return larger parent docs for context.

Problem Solved:

- Small chunks: precise retrieval, no context
- Large chunks: good context, bad precision
- Solution: retrieve by chunk, return full section

Configuration:

- Child splitter: 100-500 tokens (search units)
- Parent splitter: 1500-2000 tokens (return units)

Benefits:

- Best of both worlds
- Better context for LLM
- Improved answer quality

RAG Evaluation: Measuring Quality

Metrics Matter

Evaluate RAG across multiple dimensions for production readiness.

Retrieval Quality:

- Recall: Found all relevant docs?
- Precision: Were docs relevant?
- MRR: Rank of first match?

Overall:

- Hallucination rate
- Answer correctness

Generation Quality:

- BLEU/ROUGE: Text similarity
- Faithfulness: Grounded in context?
- Relevance: Matches query?

Cost/Speed:

- Latency (E2E)
- Tokens per query
- Dollar cost per query

RAG Fusion and Re-ranking Strategies

Beyond Simple Retrieval

Combine multiple retrieval strategies and intelligently rank results.

RAG Fusion:

- Generate multiple queries
- Retrieve from each
- Reciprocal Rank Fusion (RRF) to combine
- Deduplicate and reorder

Re-ranking Strategies:

- LLM-based: Use LLM to score docs
- Cross-encoder: Specialized ranking model
- Diversity: Cover different aspects
- Semantic: Re-rank by relevance

[Docs: langchain.retrievers]

Streaming RAG Responses

User Experience

Stream LLM response token-by-token for faster perceived latency.

How Streaming Works:

1. Retrieval happens upfront (not streamed)
2. Only LLM generation is streamed
3. Sources returned separately
4. Improve perceived responsiveness

Considerations:

- Latency still includes retrieval time
- Frontend must handle streaming
- Good for long responses

Limitation

Cannot stream documents; must retrieve first, then stream answer.

Common RAG Pitfalls

Pitfall 1: Poor Chunking

Chunks too large lose specificity, too small lack context. Fix: 500-1500 tokens, 10-20 percent overlap.

Pitfall 2: Stale Vector DB

Documents change but embeddings not updated. Fix: Re-index regularly, use delta updates.

Pitfall 3: Hallucination Despite Context

LLM ignores context or makes up facts. Fix: Grounding prompts, lower temperature.

Pitfall 4: Retrieval Failures

Query phrased differently from docs. Fix: Multi-query, hybrid search, query

RAG vs Fine-tuning: Comparison

Dimension	RAG	Fine-tune
Training	No	Yes (GPU)
Knowledge Update	Easy	Expensive
Latency	Higher	Lower
Citations	Easy	Hard
Hallucination	Reduced	Possible
Capacity	Large	Limited
Deploy Cost	Lower	Higher
Privacy	External DB	Model weights

When to Use RAG

Frequently updated knowledge, need citations, large datasets.

When to Use Fine-tuning

Task-specific behavior, latency-critical, small datasets.

Production RAG Checklist

Before Launch

Ensure robust, measurable, and maintainable system.

Data and Indexing:

- Documents loaded and versioned
- Chunking strategy tested
- Vector DB production-ready
- Embeddings model chosen

Operations:

- Caching strategy
- Rate limiting
- Error handling
- Logging and audit trail

Evaluation and Testing:

- Human eval on sample set
- Automated metrics (RAGAS)
- A/B testing framework
- Baseline established

Monitoring:

- Latency tracked
- Token costs logged
- Hallucination detection
- User feedback loop

Advanced RAG: Adaptive Routing

Routing Based on Query

Use router to decide: retrieve or answer directly? Which retriever?

Routing Strategies:

- Query classification: Does it need RAG?
- Retriever selection: Keyword vs semantic vs hybrid
- Document source routing: Which DB?
- Confidence-based: Use fallback if uncertain

Benefits:

- Reduced latency for simple queries
- Better results for complex queries
- Cost optimization

Caching in RAG: Speed and Cost

Opportunities

Cache at multiple levels: embeddings, retrievals, responses.

Query Caching:

- Cache exact same queries
- FIFO or LRU cache
- Limited by diversity

Implementation:

- In-memory cache for development
- Redis for production
- LangChain built-in support

Semantic Caching:

- Cache by query intent
- Return similar answers
- More flexible

Hybrid Search: Keyword plus Semantic

Best of Both Worlds

Combine exact keyword matching with semantic similarity.

Keyword Search:

- BM25, TF-IDF, Lucene
- Exact term matching
- Good for names/acronyms
- Fast, no embeddings

Semantic Search:

- Vector embeddings
- Synonym understanding
- Good for concepts
- Slower, requires embeddings

Approach:

- Retrieve from both
- Rank and deduplicate
- Return combined results

Metadata Filtering in RAG

Pre-filtering

Use metadata to restrict retrieval before semantic search.

Common Metadata:

- Date: Only 2025 onward
- Source: Official docs only
- Category: API reference only
- Language: English only
- Author: Peer-reviewed sources

Benefits:

- Filter irrelevant results early
- Reduce context noise
- Improve answer quality
- Lower costs

Chain of Thought RAG

Idea

Combine multi-step reasoning with iterative retrieval.

Pattern:

1. LLM breaks complex question into sub-questions
2. Retrieve context for each sub-question
3. Combine answers into final response

Example:

- Q: "Compare RAG and fine-tuning for proprietary documents"
- Sub-Q1: "What is RAG?"
- Sub-Q2: "What is fine-tuning?"
- Sub-Q3: "Advantages/disadvantages of each?"

Benefit: Better answers to complex, multi-faceted questions

Dynamic Context Window Management

Challenge

Limited context window. Too much: LLM confused. Too little: quality suffers.

Strategies:

- Adaptive K: Increase K if low confidence
- Token Budget: Reserve tokens for response
- Truncation: Summarize long docs
- Hierarchical: Summaries then full docs

Implementation:

- Monitor token usage
- Adjust K based on quality
- Track latency impact

Testing and Debugging RAG

Testing Strategy:

- Unit tests: Chunking, embeddings, indexing
- Integration tests: Retrieval accuracy, prompt formatting
- E2E tests: Full pipeline, known Q&A pairs
- Regression tests: Catch improvements and regressions

Debugging Checklist:

- Poor answers: Check retrieved docs, prompt, LLM
- Slow queries: Profile retrieval vs LLM time
- Out of memory: Reduce docs, chunk size
- Hallucinations: Improve prompts, lower temperature

Tools:

- LangSmith for tracing
- RAGAS for metrics
- Custom evaluation datasets

RAG Security: PII and Sensitive Data

Risk

RAG can expose PII if documents contain sensitive information.

Mitigation Strategies:

- PII Redaction: Remove/mask emails, SSNs before indexing
- Access Control: Only authorized users see certain docs
- Query Filtering: Restrict sensitive field retrieval
- Output Filtering: Post-process LLM output

Best Practices:

- Audit documents before indexing
- Use metadata filtering
- Monitor for data leakage
- Test with sensitive queries

RAG Cost Optimization

Budget-Conscious Approaches

Reduce embedding, LLM, and DB costs.

Optimization Strategies:

1. Smaller Models: Cheaper embeddings, smaller LLMs
2. Batch Processing: Index once, reuse embeddings
3. Reduce K: Start with 3 instead of 10
4. Caching: Cache embeddings and responses
5. Compression: Summarize docs to fit more
6. Approximate Search: HNSW, IVF vs exact

Typical Costs:

- RAG query: dollars 0.001–dollars 0.01
- Depends on model, context size, vector DB

Knowledge Graph plus RAG

Hybrid Approach

Combine semantic search with structured knowledge graphs.

Pattern:

- Extract entities and relationships from docs
- Build knowledge graph (Neo4j, RDF)
- Query graph for structured context
- Pass graph plus retrieved docs to LLM

Benefits:

- Better reasoning on relationships
- More precise, entity-based retrieval
- Combines semantic plus structured search

Multi-Modal RAG: Images plus Text

Beyond Text

Extend RAG to images, videos, audio.

Challenges:

- No standard multi-modal embeddings yet
- Slower inference, larger models
- Complex indexing (text plus images)

Approaches:

- Vision-Language Models: CLIP, LLaVA
- Hybrid Indexing: Store image embeddings and descriptions
- Image-to-Text: Captions then standard RAG

Emerging area with use case-dependent solutions.

RAG Cheat Sheet: Quick Reference

Basic RAG in 3 lines:

- `retriever = db.as_retriever(k=3)`
- `rag_chain = create_retrieval_chain(retriever, doc_chain)`
- `result = rag_chain.invoke({``input'': question})`

Key Components:

- Retriever: vector DB, keyword, multi-query
- Doc Chain: stuff, map-reduce, refine
- Full Chain: retrieval, history-aware

Production Considerations:

- Evaluate with RAGAS
- Monitor with LangSmith
- Cache aggressively
- Test thoroughly

Self-Check Questions

1. What is RAG and how does it differ from fine-tuning?
2. Explain the three steps of a RAG pipeline.
3. How does multi-query retrieval improve recall?
4. What is contextual compression and why use it?
5. How do you add chat history to RAG?
6. Name three RAG evaluation metrics.
7. What are common RAG pitfalls and fixes?
8. How can you reduce RAG costs?

Hint: Review slides on basics (frames 4-5), LCEL (frames 6-9), and evaluation (frame 18).

Summary and Next Steps

Key Takeaway

RAG is the practical way to give LLMs access to your data without retraining.

What You've Learned:

- RAG fundamentals and mental model
- Building RAG chains with LCEL
- Advanced techniques: multi-query, compression, citations
- Evaluation and production best practices
- Common pitfalls and optimization strategies

Next Steps:

1. Build a simple RAG pipeline with your documents
2. Evaluate using RAGAS or custom metrics
3. Iterate on retrieval (chunk size, K, retriever)
4. Deploy with monitoring and caching
5. Collect feedback and improve continuously

Resources and Further Reading

Official LangChain Docs:

- `https://docs.smith.langchain.com/rag`
- `https://docs.smith.langchain.com/rag-from-scratch`
- `https://api.python.langchain.com`

Evaluation and Tracing:

- RAGAS: `https://github.com/explodinggradients/ragas`
- LangSmith: `https://docs.smith.langchain.com`

Research:

- Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks (Lewis et al., 2020)
- In-Context Retrieval-Augmented Language Models (Shi et al., 2023)